Parquet Data Stores

Columnar Storage with PyArrow and pandas

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 4

1 What is Parquet?

Apache Parquet is an open, *columnar* binary file format for storing large tabular datasets. Originally developed at Twitter and Cloudera for the Hadoop ecosystem, it is now the default storage format for data lakes (Spark, DuckDB, Polars, Snowflake, BigQuery, AWS Athena) and is widely used for market data.

- **Columnar layout.** CSV files and most databases store data *row by row*. Parquet stores all values of one column contiguously. A query that needs only `price` and `shares` reads only those two columns from disk — the other columns are never touched. Since most analytics use a handful of columns out of many, this is the main source of Parquet's speed.
- **Row groups, column chunks, pages.** A file is split horizontally into *row groups* (typically 10^5–10^6 rows). Within a row group, each column is stored as a *column chunk*, which is further divided into *pages* (the unit of encoding and compression). The row group is the smallest unit that is read independently.
- **Self-describing schema and statistics in the footer.** The file footer holds the schema (column names and types), the location of every column chunk, and per-chunk *statistics*: min, max, and null count. Readers use these statistics to skip entire row groups that cannot match a filter — *predicate pushdown* — without decompressing them.
- **Encoding plus compression.** Because a column is homogeneous, Parquet first applies a lightweight *encoding* — dictionary encoding for repeated values (tickers, flags), run-length encoding, delta encoding for sorted integers such as timestamps — and then a general-purpose *compressor* (Snappy, ZSTD, GZIP, LZ4). Columnar data compresses much better than row-oriented data, often 2–10× smaller than the equivalent CSV.
- **Immutable files.** A Parquet file is written once and never modified in place: you cannot append rows to an existing file. "Appending" means writing *another file* into the same directory, which is why Parquet data is normally organized as a *dataset* — a directory of files (Section 3).

2 PyArrow: The Direct Python Interface

PyArrow is the Python binding to Apache Arrow, an in-memory columnar format whose layout mirrors Parquet's. `pyarrow.parquet` reads and writes Parquet files to and from Arrow `Table` objects; pandas, Polars, and DuckDB all use it underneath. Use PyArrow directly when you need control over row groups, compression, schema metadata, or streaming writes.

- **Arrow tables and types.** `pa.table({...})` or `pa.Table.from_pandas(df)` builds a columnar in-memory table. Arrow types are explicit: `int8`, `uint32`, `float64`, `string`, `timestamp[ns]`, `duration[ns]` (the Arrow equivalent of `timedelta64[ns]`), and `dictionary<string>` (the equivalent of a pandas `category`).
- **Writing.** `pq.write_table(table, path, compression='zstd', row_group_size=...)` writes a single file. For data that does not fit in memory, `pq.ParquetWriter` writes one batch at a time into the same file, each batch becoming one or more row groups; the file is finalized when the writer is closed.
- **Column projection.** `pq.read_table(path, columns=['timestamp', 'price'])` reads only the listed columns; the others are never read from disk.
- **Row filters (predicate pushdown).** `filters=[('price', '>', 150.0)]` skips every row group whose `max(price)` ≤ 150 using the footer statistics, then filters the remaining rows exactly. Filters are effective only when the data is *sorted or clustered* on the filtered column; on randomly ordered data every row group's min/max range covers the predicate, and nothing is skipped.
- **Metadata.** `pq.ParquetFile(path)` opens the footer without reading data. It exposes the schema (`.schema_arrow`), the row and row-group counts (`.metadata.num_rows`, `.metadata.num_row_groups`), and per-chunk statistics (`.metadata.row_group(i).column(j).statistics`). File-level key–value metadata

(units, source, date) is attached with `table.replace_schema_metadata({...})`.

```python
import pyarrow as pa, pyarrow.parquet as pq, pandas as pd

table = pa.table({
    'timestamp': pa.array(pd.to_timedelta(['09:30:00', '09:30:01', '09:30:02'])),
    'price'    : pa.array([150.10, 150.12, 150.09], pa.float64()),
    'volume'   : pa.array([100,    200,    150   ], pa.int32()),
    'stock'    : pa.array(['AAPL'] * 3).dictionary_encode(),   # categorical
})
table = table.replace_schema_metadata({'source': 'vendor feed',
                                       'date'  : '2024-01-02'})  # file metadata

# --- Write ---
pq.write_table(table, 'example.parquet',
               compression='zstd', row_group_size=1_000_000)

# --- Inspect the footer (no data is read) ---
pf = pq.ParquetFile('example.parquet')
print(pf.schema_arrow)                                # timestamp: duration[ns] ...
print(pf.metadata.num_row_groups)                     # 1
print(pf.metadata.row_group(0).column(1).statistics)  # min: 150.09  max: 150.12

# --- Read: two columns only, with a row filter ---
t  = pq.read_table('example.parquet',
                   columns=['timestamp', 'price'],
                   filters=[('price', '>', 150.095)])
df = t.to_pandas()                                    # timestamp -> timedelta64[ns]
```

3 Partitioned Datasets and pandas

A **dataset** is a directory tree of Parquet files that share a schema. With *Hive-style partitioning*, the directory names encode column values: `trades/stock=AAPL/part-0.parquet`, `trades/stock=MSFT/part-0.parquet`. The partition column is not stored inside the files; it is reconstructed from the path.

- **Partition pruning.** A filter on a partition column (`stock == 'AAPL'`) eliminates whole directories before any file is opened: non-matching files are never touched.
- **Choosing partition columns.** Partition on low-cardinality columns that queries filter on almost always — trading date, message type, perhaps symbol. Avoid partitions that produce thousands of tiny files: each file has fixed open/footer overhead, and files under ∼10 MB are inefficient. Within a partition, *sort* by the next most common filter column (e.g. an instrument ID, then `timestamp`) so row-group statistics can do the rest.
- **pandas interface.** `df.to_parquet(path, partition_cols=[...])` and `pd.read_parquet(path, columns=..., filters=...)` wrap PyArrow. Strings are variable-length, and repeated strings are dictionary-encoded automatically. Partition columns come back as `category` dtype.
- **`pyarrow.dataset`.** For larger-than-memory work, `ds.dataset(path, partitioning='hive')` returns a lazy dataset object; `.to_table(columns=..., filter=ds.field('stock') == 'MSFT')` or `.to_batches()` stream only the needed data. DuckDB and Polars can query the same directory with SQL or lazy expressions without loading it into pandas.

3.1 Minimal working example

```python
import pandas as pd, numpy as np

# --- Build a small trade DataFrame ---
df = pd.DataFrame({
    'timestamp': pd.to_timedelta(['09:30:00', '09:30:01', '09:30:02', '09:30:03']),
    'price'    : [150.10, 150.12, 150.09, 410.50],
    'volume'   : [100, 200, 150, 50],
    'side'     : np.array([1, -1, 1, -1], dtype='int8'),  # 1 = buy, -1 = sell
    'stock'    : ['AAPL', 'AAPL', 'AAPL', 'MSFT'],
})

```

```python
# --- Write a dataset partitioned by stock ---
df.to_parquet('trades', partition_cols=['stock'],
              compression='zstd', index=False)
# trades/stock=AAPL/<uuid>-0.parquet
# trades/stock=MSFT/<uuid>-0.parquet

# --- Read everything back ---
all_trades = pd.read_parquet('trades')        # 'stock' returns as category

# --- Pushdown query: AAPL buys only, two columns ---
aapl_buys = pd.read_parquet('trades',
                            columns=['timestamp', 'price'],
                            filters=[('stock', '==', 'AAPL'), ('side', '==', 1)])
```

Key observations:

- Calling `to_parquet` again on the same directory adds new files to each partition rather than modifying existing ones — this is how "appending" works.
- The `stock` filter prunes the `MSFT` directory without opening it, while the `side` filter is applied using row-group statistics and then row by row.
- `timedelta64[ns]` round-trips exactly through Arrow's `duration[ns]` type, preserving nanosecond precision.

4 Patterns for Large Datasets

- **One dataset per record type.** A dataset has one schema; keep record types with different fields apart.
- **Sort before writing.** Parquet has no secondary index. If you usually retrieve one instrument at a time, sort by its ID (then by time) before writing: each row group then covers a narrow range of IDs, and `filters=[('id', '==', 7)]` skips nearly all of them. (A streamed file, below, is in arrival order; sort and rewrite it afterwards.)
- **Use narrow types.** `int8` for signs and flags, `int32` or `uint16` for IDs, `float32` where precision allows. Encode short categorical codes as small integers or let Parquet dictionary-encode them.
- **Store times as `timestamp[ns]`** for nanosecond precision.
- **Stream in batches.** `pq.ParquetWriter` writes one batch at a time to the same file, each batch becoming a row group. Peak memory is set by the batch size, not by the file size. Declare the schema up front so every batch matches.

```python
import pyarrow as pa, pyarrow.parquet as pq

schema = pa.schema([('time',  pa.timestamp('ns')),
                    ('id',    pa.int32()),
                    ('value', pa.float64())])
BATCH = 250_000
buffer = []
with pq.ParquetWriter('events.parquet', schema, compression='zstd') as writer:
    for rec in records():                      # any iterator of dicts
        buffer.append(rec)
        if len(buffer) == BATCH:
            writer.write_table(pa.Table.from_pylist(buffer, schema=schema))
            buffer.clear()
    if buffer:                                 # final partial batch
        writer.write_table(pa.Table.from_pylist(buffer, schema=schema))
# Read back in bounded pieces
for batch in pq.ParquetFile('events.parquet').iter_batches(batch_size=100_000):
    ...                                        # batch is a pyarrow.RecordBatch
```

The takeaway. Parquet's speed comes from reading only what you need: columns via `columns=`, directories via partition pruning, and row groups via footer statistics — which only work when the data are sorted or clustered on the filtered column.